Chijandas Grocery Mall

Technical Interview Preparation Guide

---

1. The 30-Second Elevator Pitch

"I built Chijandas Grocery Mall, a full-stack inventory management and customer storefront application using the MERN stack (MongoDB, Express, React, and Node.js). It features a secure administrative dashboard with live charts (Recharts) to track revenue and stock levels, alongside a customer catalog where users can order items. A key highlight is the system's resilience: I implemented a database adapter pattern that automatically falls back to local JSON file storage if the database server goes offline, ensuring 100% uptime for critical operations."

2. Core Tech Stack

- Frontend: React (Vite), CSS3, Recharts, Lucide React, EmailJS SDK
- Backend: Node.js, Express.js, JSON Web Tokens (JWT), bcryptjs
- Database: MongoDB (Atlas Cloud + Local) with JSON Fallback Abstraction

3. Key Technical Talking Points

• Robust Dual-Database Fallback Mode

On server startup, the database manager automatically attempts a connection to MongoDB. If the DB server is offline, it gracefully falls back to a JSON-based database (fallback_db.json) without crashing, maintaining identical REST API endpoint structures.

• Role-Based Access Control (RBAC) & Route Protection

User accounts are strictly separated (admin vs user). API endpoints are protected using custom JWT verification middleware, and passwords are encrypted using bcryptjs before storage.

• Indian Pincode Auto-Fill Integration

Entering a 6-digit Pincode triggers an asynchronous call to the Indian Postal API, auto-detecting and pre-filling the customer's City and State using a custom fuzzy matching script.

• Transactional Email Notification System (EmailJS)

Client-side triggers fire two concurrent email operations on checkout completion: a customer invoice receipt and a separate high-priority admin alert template.

4. Common Interview Q&A

Q1: Why did you choose MongoDB instead of a Relational Database like MySQL?

Answer: "I chose MongoDB because of its flexible document-based structure. In a grocery mall catalog,

products can have highly dynamic attributes (e.g. expiration dates for food products vs warranties for kitchen utensils). Storing products as flexible documents fits this model much better than rigid SQL tables. Additionally, MongoDB Atlas provides a free, highly scalable cloud tier which made hosting the production database seamless."

Q2: What was the most challenging part of this project, and how did you solve it?

Answer: "The most challenging part was ensuring the backend API routes remained identical regardless of whether the system was running in MongoDB mode or fallback JSON mode. To solve this, I designed a unified dbManager interface that exposes standard database actions (like findOne, create, and findByIdAndUpdate). The rest of the API routes simply call dbManager, completely unaware of the underlying database type. This kept the router files extremely clean and modular."

Q3: How did you handle security in the authentication flow?

Answer: "On the backend, user passwords are never stored in plaintext—they are hashed using bcrypt before database insertion. For sessions, I chose JWT tokens stored in the browser's localStorage and sent in the Authorization header. On the API side, I created a custom middleware that validates the JWT token, fetches the user's role, and authorizes admin-only requests (like adding or deleting products) while rejecting unauthorized clients with a 403 Forbidden status."

5. Future Roadmap & Enhancements

1. Cart Checkout System: Expand from single-item checkouts to a multi-product shopping cart.
2. Online Payment Gateway: Integrate Stripe or Razorpay alongside UPI/COD.
3. Automated Stock Alerts: Connect Twilio or Slack API to notify the administrator of low inventory levels.